7/26/2026

[ShadowFox Data Science Internship]

[Intermediate Level Task]

[Air Quality Index (AQI) Analysis Using Python]

priyanshu1117r@hotmail.com

Introduction:

Air pollution is one of the most critical environmental issues affecting human health and ecosystems worldwide. Delhi, the capital city of India, frequently experiences poor air quality due to rapid urbanization, industrial emissions, vehicle exhaust, construction activities, and seasonal crop residue burning. Monitoring air pollution is essential for understanding its impact and developing effective environmental policies.

The Air Quality Index (AQI) is a standardized indicator that measures the overall quality of air based on the concentration of pollutants such as PM2.5, PM10, NO_2, SO_2, CO, and Ozone. This project analyses the Delhi AQI dataset using Python to explore pollution trends, visualize pollutant distributions, and identify relationships among different air quality parameters.

Objectives:

The main objectives of this project are:

- Analysis the Delhi AQI dataset.
- Perform data cleaning and preprocessing.
- Identify missing values and data quality issues.
- Explore pollutant distributions using visualizations.
- Study relationships among different pollutants.
- Understand AQI trends over time.
- Generate meaningful insights from the data.

Import Required Libraries:

The required Python libraries are imported to perform data manipulation, numerical computations, and visualization**. Pandas** is used for handling datasets, **NumPy** for numerical operations, **Matplotlib** and **Seaborn** for creating informative graphs.

Code Snippet:

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
```

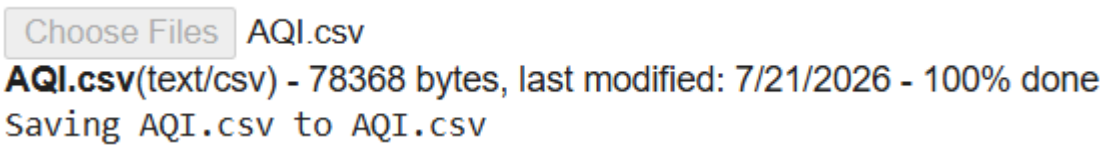
Upload Google files Dataset:

The dataset is loaded into a Pandas Data Frame for further analysis.

Code Snippet:

```
from google.colab import files
uploaded = files.upload()
```

Output:



Load the Dataset and load first five Rows:

The dataset is loaded into a Pandas Data Frame for further analysis.

Code Snippet:

```
import pandas as pd
df = pd.read_csv("AQI.csv")
df.head()
```

Output:

	Date	Month	Year	Holidays_Count	Days	PM2.5	PM10	NO2	SO2	CO	Ozone	AQI
0	1	1	2021	0	5	408.80	442.42	160.61	12.95	2.77	43.19	462
1	2	1	2021	0	6	404.04	561.95	52.85	5.18	2.60	16.43	482
2	3	1	2021	1	7	225.07	239.04	170.95	10.93	1.40	44.29	263
3	4	1	2021	0	1	89.55	132.08	153.98	10.42	1.01	49.19	207
4	5	1	2021	0	2	54.06	55.54	122.66	9.70	0.64	48.88	149

Load the Dataset last Five Rows:

The last five rows of the dataset were displayed using the tail() function. This step helps verify the successful loading of the dataset and provides a quick overview of the final records, ensuring that the data is complete and ready for analysis.

Code Snippet:

```
df.tail()
```

Output:

	Date	Month	Year	Holidays_Count	Days	PM2.5	PM10	NO2	SO2	CO	Ozone	AQI
1456	27	12	2024	0	5	58.43	249.17	41.69	65.89	0.99	36.25	263
1457	28	12	2024	0	6	33.83	150.77	33.31	66.14	0.79	35.19	113
1458	29	12	2024	1	7	31.21	139.75	27.01	65.94	0.57	35.88	142
1459	30	12	2024	0	1	38.01	152.83	29.12	65.16	0.55	38.38	116
1460	31	12	2024	0	2	80.42	318.96	40.37	64.98	0.84	39.93	209

Dataset Overview:

Explanation:

This step provides basic information about the dataset, including column names, data types, number of records, and statistical summary.

1. **Code Snippet:**

```
df.shape
```

Output:

```
(1461, 12)
```

2. **Code Snippet:**

```
df.columns
```

Output:

```
Index(['Date', 'Month', 'Year', 'Holidays_Count', 'Days', 'PM2.5', 'PM10',
       'NO2', 'SO2', 'CO', 'Ozone', 'AQI'],
      dtype='object')
```

3. **Code Snippet:**

```
df.info()
```

Output:

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1461 entries, 0 to 1460
Data columns (total 12 columns):
 #   Column          Non-Null Count  Dtype
---  ------          --------------  -----
 0   Date            1461 non-null   int64
 1   Month           1461 non-null   int64
 2   Year            1461 non-null   int64
 3   Holidays_Count  1461 non-null   int64
 4   Days            1461 non-null   int64
 5   PM2.5           1461 non-null   float64
 6   PM10            1461 non-null   float64
 7   NO2             1461 non-null   float64
 8   SO2             1461 non-null   float64
 9   CO              1461 non-null   float64
 10  Ozone           1461 non-null   float64
 11  AQI             1461 non-null   int64
dtypes: float64(6), int64(6)
memory usage: 137.1 KB
```

Explanation:

The info() function was used to examine the structure of the dataset. It provides details about the number of records, column names, data types, non-null values, and memory usage. This step is useful for understanding the dataset and preparing it for further data analysis and preprocessing.

4. **Code Snippet:**

```
df.describe()
```

Output:

	Date	Month	Year	Holidays_Count	Days	PM2.5	PM10	NO2	SO2	CO	Ozone	AQI
count	1461.000000	1461.000000	1461.000000	1461.000000	1461.000000	1461.000000	1461.000000	1461.000000	1461.000000	1461.000000	1461.000000	1461.000000
mean	15.729637	6.522930	2022.501027	0.189596	4.000684	90.774538	218.219261	37.184921	20.104921	1.025832	36.338871	202.210815
std	8.803105	3.449884	1.118723	0.392116	2.001883	71.650579	129.297734	35.225327	16.543659	0.608305	18.951204	107.801076
min	1.000000	1.000000	2021.000000	0.000000	1.000000	0.050000	9.690000	2.160000	1.210000	0.270000	2.700000	19.000000
25%	8.000000	4.000000	2022.000000	0.000000	2.000000	41.280000	115.110000	17.280000	7.710000	0.610000	24.100000	108.000000
50%	16.000000	7.000000	2023.000000	0.000000	4.000000	72.060000	199.800000	30.490000	15.430000	0.850000	32.470000	189.000000
75%	23.000000	10.000000	2024.000000	0.000000	6.000000	118.500000	297.750000	45.010000	26.620000	1.240000	45.730000	284.000000
max	31.000000	12.000000	2024.000000	1.000000	7.000000	1000.000000	1000.000000	433.980000	113.400000	4.700000	115.870000	500.000000

Explanation:

The describe() function was used to obtain a statistical summary of the numerical data in the dataset. It provides important measures such as the mean, minimum, maximum, standard deviation, and quartile values, which help in understanding the overall distribution and characteristics of the data before performing further analysis.

Checking Missing Values:

Explanation:

The isnull().sum() function was used to identify missing values in each column of the dataset. Detecting missing values is an essential data preprocessing step, as it helps ensure data quality and improves the accuracy of further analysis and visualization.

Code Snippet:

```
df.isnull().sum()
```

Output:

	0
Date	0
Month	0
Year	0
Holidays_Count	0
Days	0
PM2.5	0
PM10	0
NO2	0
SO2	0
CO	0
Ozone	0
AQI	0

dtype: int64

Removing Duplicate Records:

Code Snippet:

```
df.drop_duplicates(inplace=True)
```

Explanation:

The drop_duplicates(inplace=True) function was used to remove duplicate records from the dataset. This step improves data quality by ensuring that

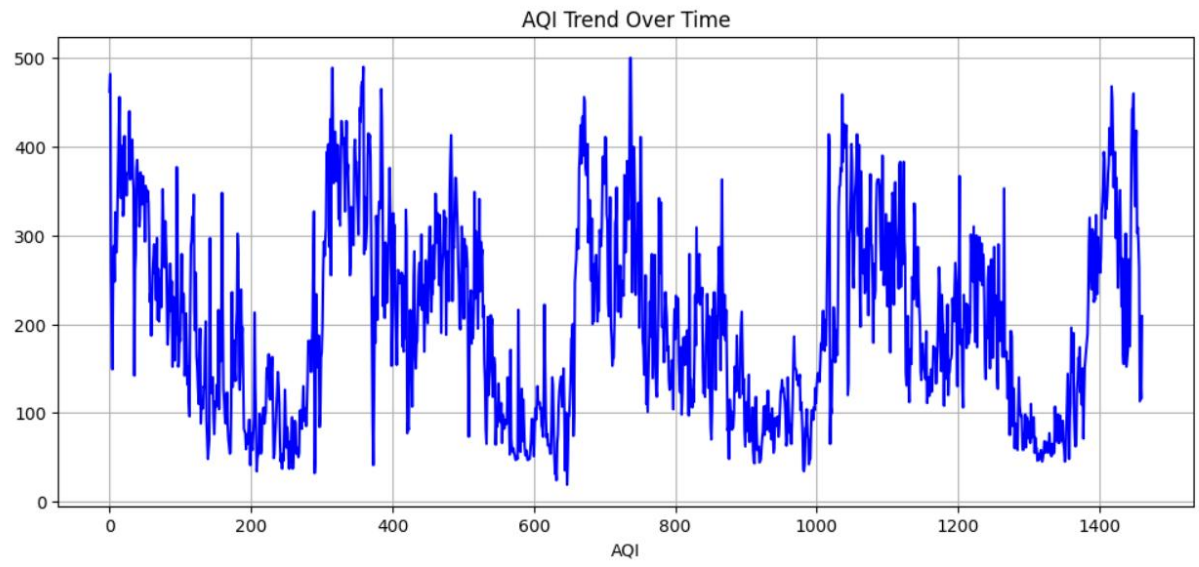
each observation is unique, leading to more accurate analysis and reliable results.

AQI Trend Over Time (Line Plot):

Code Snippet:

```
import matplotlib.pyplot as plt
plt.figure(figsize=(12,5))
plt.plot(df["AQI"], color="blue")
plt.title("AQI Trend Over Time")
plt.xlabel("AQI")
plt.grid(True)
plt.show()
```

Output:



Explanation:

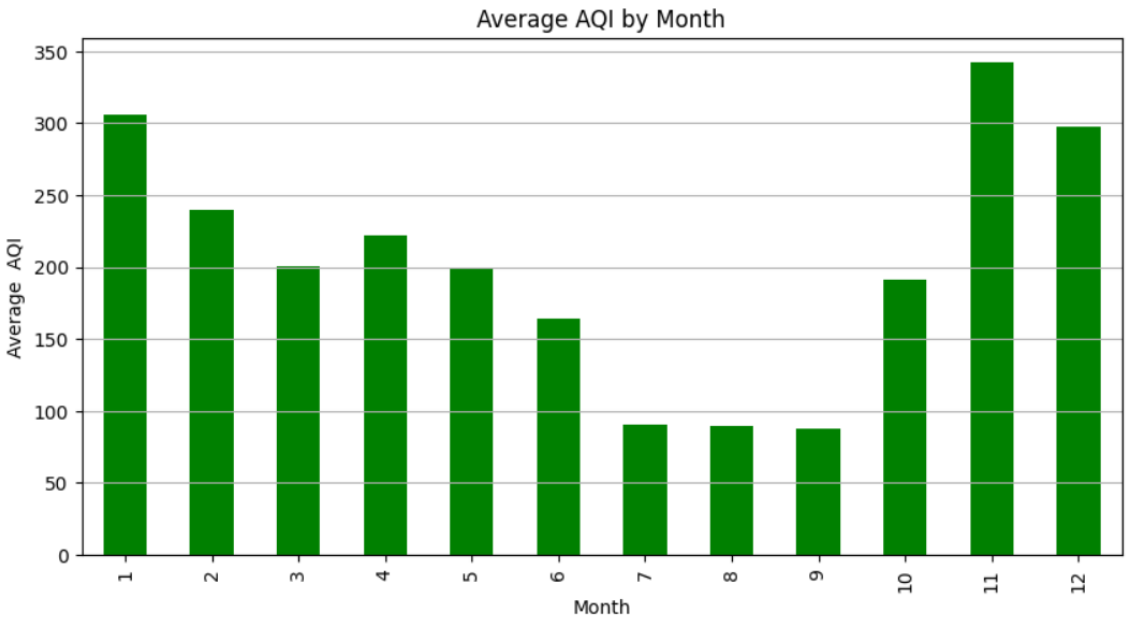
The line plot illustrates the trend of Air Quality Index (AQI) values in the dataset. It helps visualize fluctuations and overall changes in air quality, making it easier to identify periods of higher or lower pollution levels.

Average AQI by Month (Bar Chart):

Code Snippet:

```
plt.figure(figsize=(10,5))
df.groupby("Month")["AQI"].mean().plot(kind="bar", color="green")
plt.title("Average AQI by Month")
plt.xlabel("Month")
plt.ylabel("Average  AQI")
plt.grid(axis="y")
plt.show()
```

Output:



Explanation:

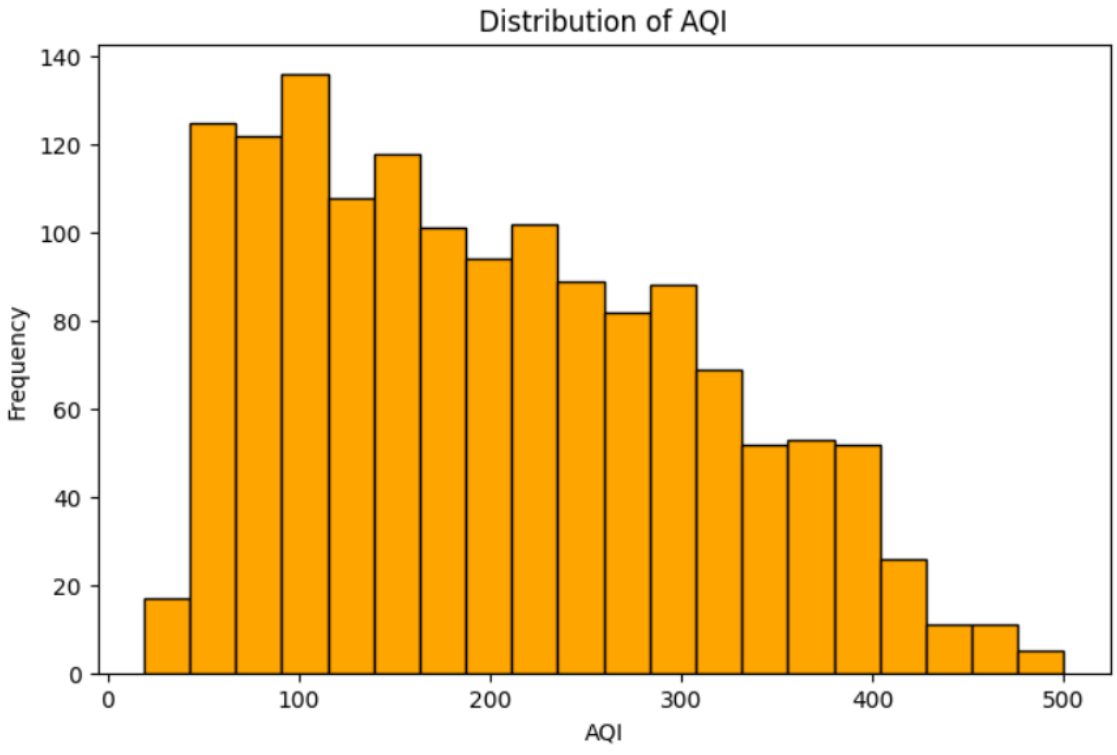
The bar chart shows the **average AQI for each month**, allowing a comparison of air quality throughout the year. It helps identify the months with higher or lower pollution levels and provides insights into seasonal variations in air quality.

Distribution of AQI (Histogram):

Code Snippet:

```
plt.figure(figsize=(8,5))
plt.hist(df["AQI"], bins=20, color="orange", edgecolor="black")
plt.title("Distribution of AQI")
plt.xlabel("AQI")
plt.ylabel("Frequency")
plt.show()
```

Output:



Explanation:

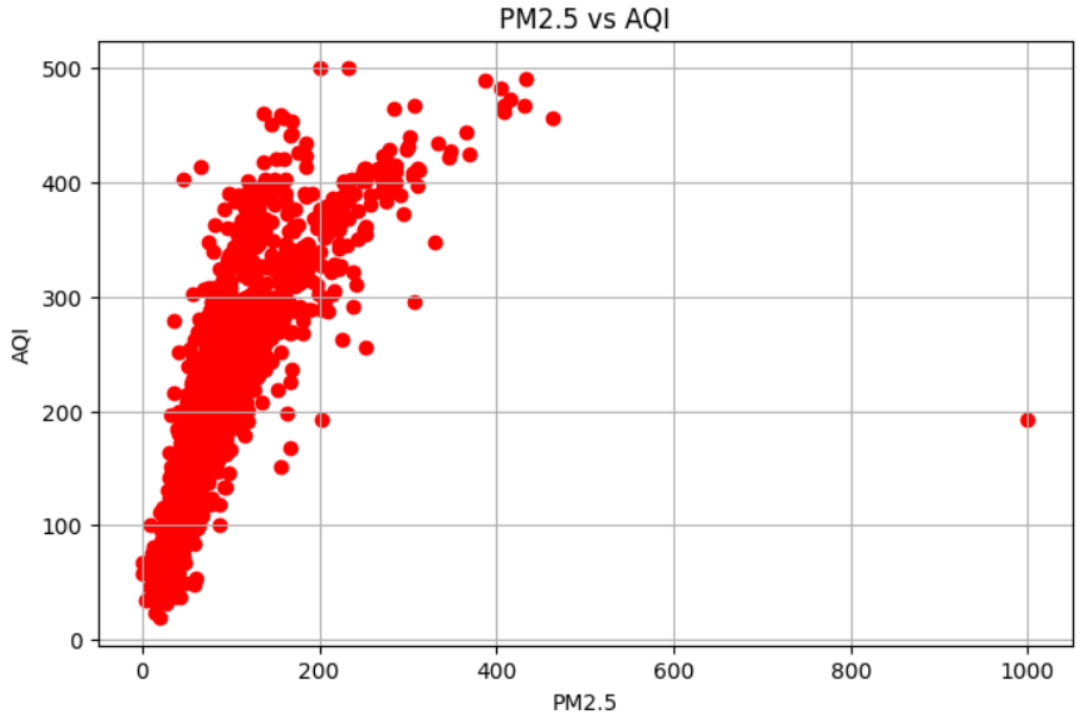
The histogram illustrates the frequency distribution of AQI values in the dataset. It helps identify the most common AQI ranges, understand the spread of air quality measurements, and detect any unusual or extreme values that may exist in the data.

PM2.5 vs AQI (Scatter Plot):

Code Snippet:

```
plt.figure(figsize=(8,5))
plt.scatter(df["PM2.5"], df["AQI"], color="red")
plt.title("PM2.5 vs AQI")
plt.xlabel("PM2.5")
plt.ylabel("AQI")
plt.grid(True)
plt.show()
```

Output:



Explanation:

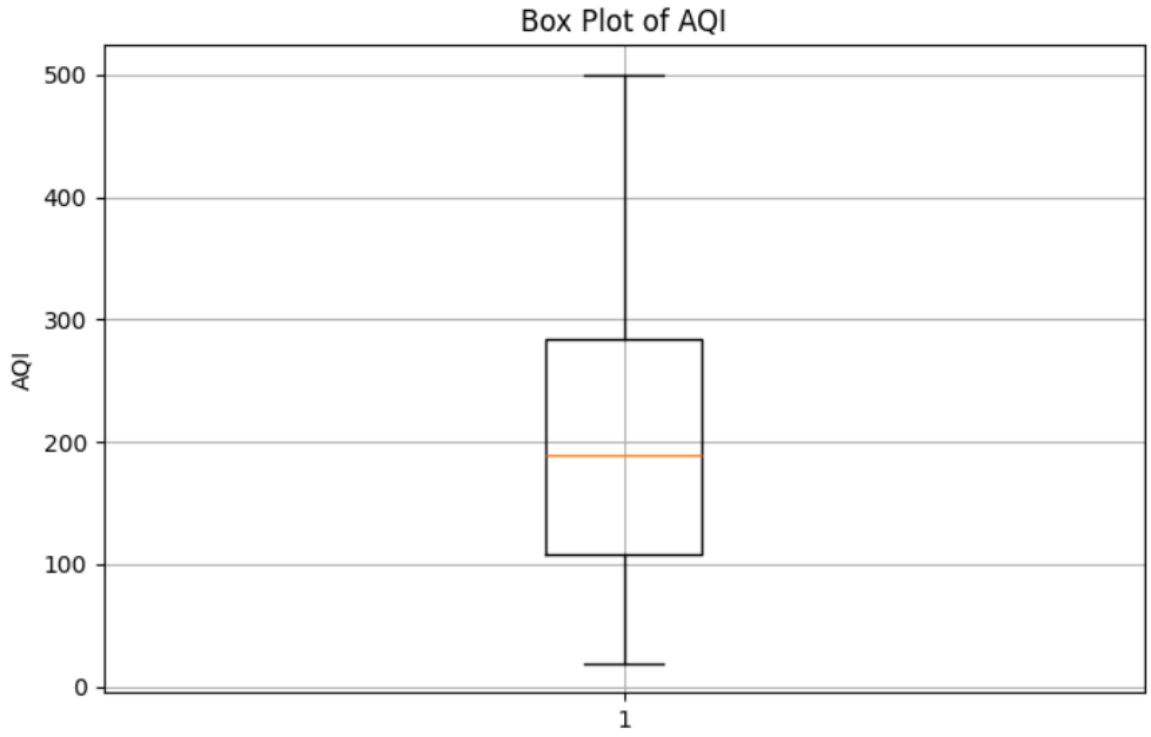
The scatter plot illustrates the relationship between PM2.5 concentration and the Air Quality Index (AQI). It helps determine whether higher PM2.5 levels are associated with higher AQI values, providing insights into the impact of particulate matter on overall air quality.

Box Plot of AQI:

Code Snippet:

```
plt.figure(figsize=(8,5))
plt.boxplot(df["AQI"])
plt.title("Box Plot of AQI")
plt.ylabel("AQI")
plt.grid(True)
plt.show()
```

Output:



Explanation:

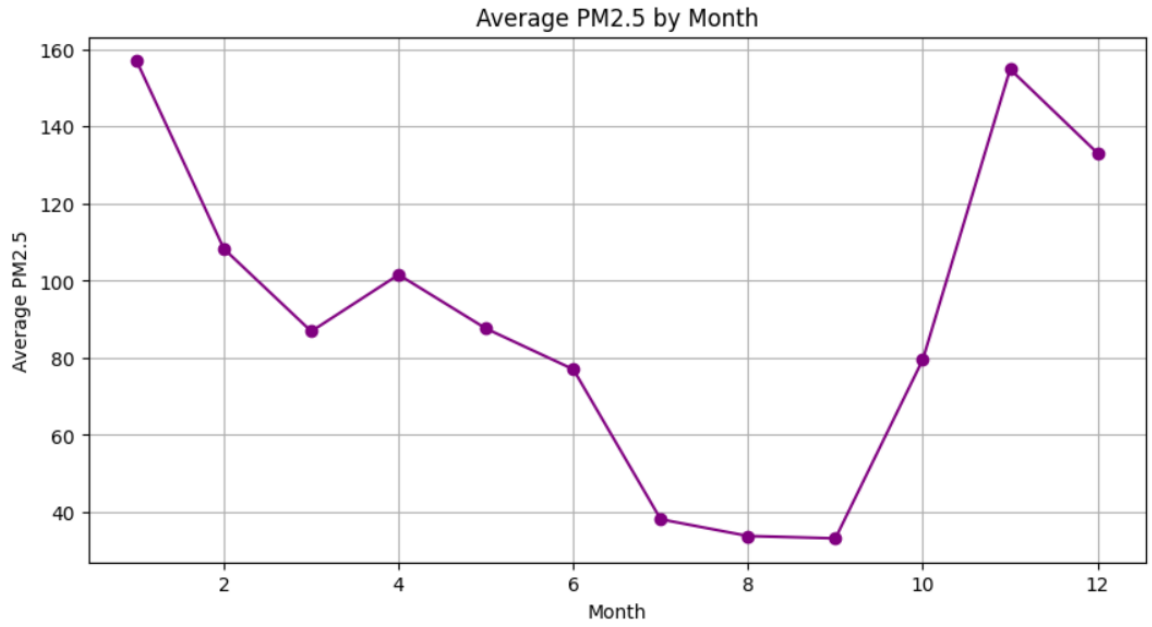
The box plot summarizes the distribution of AQI values and highlights any outliers present in the dataset. It provides a clear understanding of data variability and central tendency.

Average PM2.5 by Month:

Code Snippet:

```
plt.figure(figsize=(10,5))
df.groupby("Month")["PM2.5"].mean().plot(kind="line", marker="o", color="purple")
plt.title("Average PM2.5 by Month")
plt.xlabel("Month")
plt.ylabel("Average PM2.5")
plt.grid(True)
plt.show()
```

Output:



Explanation:

he line plot illustrates the monthly average PM2.5 concentration. It helps analyze seasonal variations in particulate pollution and identify periods with relatively poor air quality.

Heatmap (Correlation Matrix):

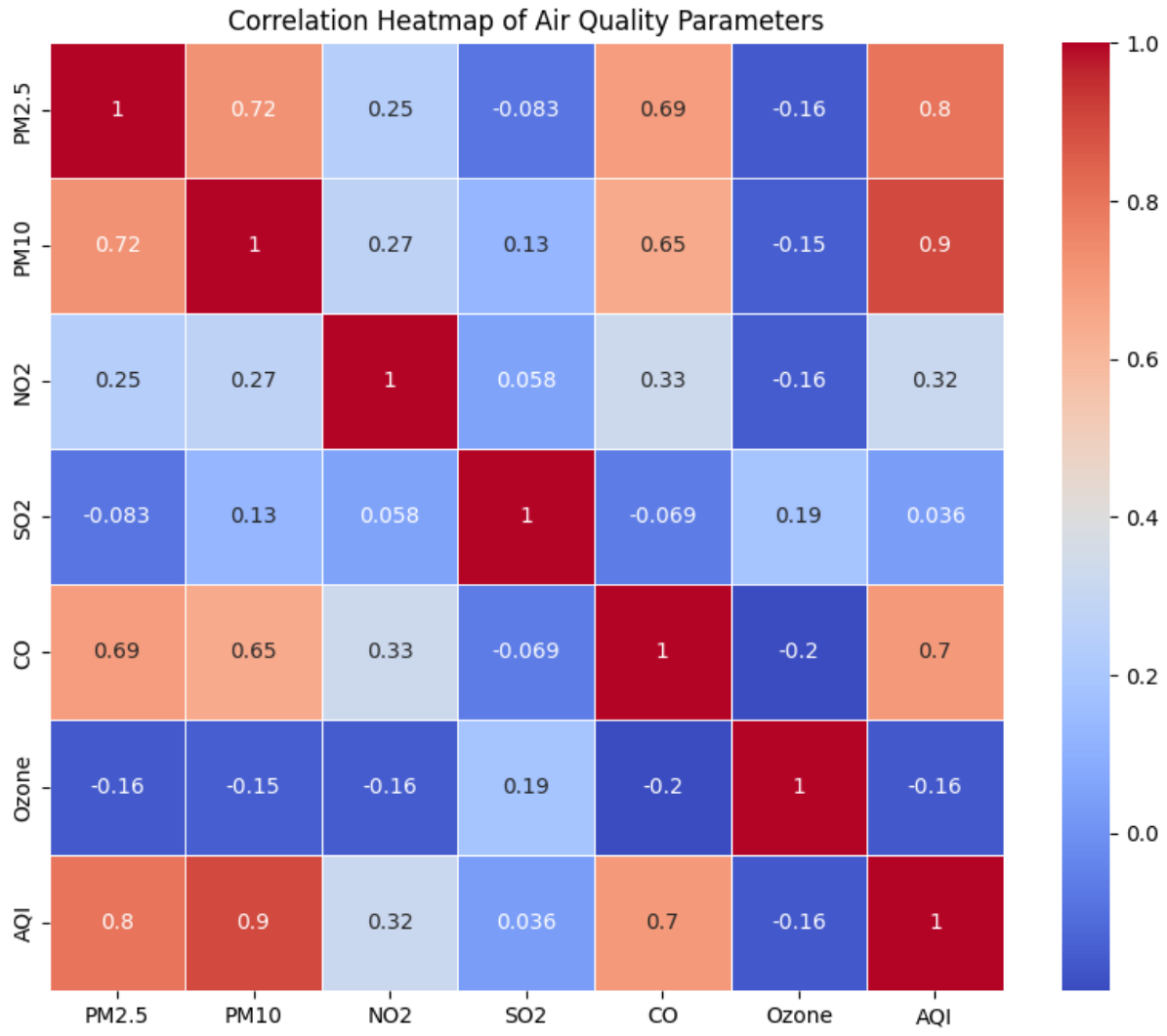
Explanation:

The correlation heatmap displays the relationship between different air quality parameters using color intensity and correlation coefficients. It helps identify which pollutants are strongly related to each other and how they collectively influence the Air Quality Index (AQI).

Code Snippet:

```
import seaborn as sns
import matplotlib.pyplot as plt
plt.figure(figsize=(10, 8))
numerical_cols = ['PM2.5', 'PM10', 'NO2', 'SO2', 'CO', 'Ozone', 'AQI']
correlation_matrix = df[numerical_cols].corr()
sns.heatmap(correlation_matrix,
            annot=True,
            cmap='coolwarm',
            linewidths=0.5)
plt.title("Correlation Heatmap of Air Quality Parameters")
plt.show()
```

Output:



Key Insights:

. The dataset was successfully cleaned and analyzed using Python.

. PM2.5 and PM10 showed noticeable variations over time, indicating changing pollution levels.

. Correlation analysis revealed positive relationships between several pollutants and AQI.

. Higher concentrations of pollutants generally resulted in higher AQI values.

. Time-series analysis highlighted fluctuations in air quality across different dates.

. Visualizations helped identify pollutant trends, distributions, and relationships effectively.

. The analysis demonstrates the importance of monitoring multiple pollutants to understand overall air quality.

Conclusion:

This project successfully analyzed the Delhi Air Quality dataset using Python. Data preprocessing, exploratory data analysis, and visualization techniques helped understand pollution patterns and relationships among different pollutants. The findings indicate that pollutants such as PM2.5, PM10, NO_2, SO_2, CO, and Ozone significantly influence the Air Quality Index (AQI). This project demonstrates the practical application of data analysis techniques for environmental monitoring and supports better decision-making for air quality management.

Future Scope:

. Develop machine learning models to predict AQI.

. Build an interactive dashboard using Streamlit or Power BI.

. Analyze seasonal variations in air pollution.

. Compare air quality across different cities.

. Integrate real-time air quality data using APIs.

References:

. Python Documentation

. Pandas Documentation

. NumPy Documentation

. Matplotlib Documentation

. Seaborn Documentation

. Delhi Air Quality Dataset (AQI.csv)